

Міністерство освіти і науки України
Львівський національний університет імені Івана Франка
Факультет електроніки і комп'ютерних технологій

Звіт
про виконання лабораторної роботи №4
з курсу «Хмарні технології та великі дані»
на тему:
«Throttle (Дросель)»

Виконав:
студент групи ФсІ-42
Трофимов Володимир
Перевірив:
ас. Керод Т. І.

Зміст

Тема.....	3
Мета	3
Вхідні дані	3
Завдання.....	4
Критерії виконання роботи.....	4
Теоретичні відомості	4
Структура програмної реалізації	5
Тестування програмної реалізації.....	5
Висновок	8
Код програми	9

Тема

Обмеження інтенсивності навантаження та стабілізація потоку викликів за допомогою патерну Throttle (Дросель).

Мета

Дослідження принципу функціонування патерну Throttle (Дросель). Розробка обгортки над функціями мовою TypeScript для контролю частоти виконання викликів із підтримкою двох моделей лімітування (фіксоване/ковзне вікно та «Token Bucket»), гнучких режимів виконання (leading, trailing), різних стратегій обробки надлишку (відсікання drop та черга FIFO), а також інструментів життєвого циклу (cancel, flush). Перевірка коректності роботи за допомогою взаємодії через HTTP-запити.

Вхідні дані

Параметр	Значення	Призначення
rateLimit	{limit, intervalMs} або {capacity, refillRate, refillIntervalMs}	Конфігурація моделі обмеження: часове вікно (N викликів за інтервал) або алгоритм кошика маркерів (Token Bucket).
limit	Ціле додатне число (наприклад, 3)	Кількість дозволених виконань у межах вказаного вікна.
intervalMs	Число в мс (наприклад, 1000 мс)	Тривалість часового вікна контролю.
capacity	Додатне число (наприклад, 3)	Максимальна місткість кошика (базовий запас викликів).
refillRate	Додатне число (наприклад, 1)	Швидкість поповнення маркерів за період поповнення.
refillIntervalMs	Число в мс (за замовчуванням 1000 мс)	Періодичність поповнення кошика токенами.
leading	boolean (за замовчуванням true)	Чи виконувати виклик негайно на початку інтервалу (leading edge).
trailing	boolean (за замовчуванням true)	Чи виконувати виклик після завершення кулдауну з останніми переданими аргументами (trailing edge).
overflowStrategy	'drop' 'queue' (за замовчуванням 'drop')	Стратегія при вичерпанні ліміту: скидання зайвих запитів (drop) або буферизація у чергу FIFO (queue).

Завдання

- Розробити універсальну функцію вищого порядку `throttle<T>(fn, rateLimit, options?)` мовою TypeScript зі строгим збереженням типів аргументів та значень, що повертаються.
- Забезпечити підтримку двох алгоритмів лімітування:
 - Виконне обмеження (sliding window) за кількістю викликів за мілісекунди.
 - Алгоритм Token Bucket із плавним поповненням токенів у часі.
- Реалізувати конфігуровані фази виконання (leading, trailing) та стратегії переповнення (drop, queue).
- Додати методи керування станом: `.cancel()` (очищення черги й таймерів) та `.flush()` (негайний форсований випуск запланованих задач).
- Реалізувати середовище автономної ручної перевірки без сторонніх тестових бібліотек (через демонстраційний скрипт та HTTP-сервер).

Критерії виконання роботи

- Коректність віконного обмеження: при серії з 10 викликів за 1 секунду та ліміті 3 виклики/с виконується рівно 3 операції, а 7 зайвих відсікаються (при `overflowStrategy: 'drop'`).
- Передбачуваність граничних фаз:
 - У режимі `leading: true, trailing: false` виклик спрацьовує одразу, подальші спами ігноруються.
 - У режимі `leading: false, trailing: true` початковий виклик відкладається, а після очікування виконується виклик з найсвіжішим станом аргументів.
- Збереження черги (FIFO): при стратегії `'queue'` всі передані виклики виконуються в строгому порядку надходження без перевищення ліміту частоти.
- Керування життєвим циклом: виклик `.cancel()` гарантовано зупиняє виконання відкладених операцій та спустошує внутрішню чергу.

Теоретичні відомості

Throttle (Дросель) — структурний та поведінковий патерн проєктування систем, покликаний забезпечити регулярність виконання операцій не частіше, ніж раз на заданий проміжок часу.

На відміну від патерну Debounce (який чекає періоду тиші перед виконанням дії), Throttle гарантує періодичне спрацьовування під час тривалого безперервного потоку подій.

Основні механізми обмеження:

- Часове вікно (Sliding Window Log / Rate Limiting): відстежує часові мітки останніх викликів. Запити старші за поточне вікно вилучаються з обліку.

Якщо кількість викликів у межах інтервалу досягає ліміту, подальші операції блокуються до вивільнення слота часу.

- Кошик токенів (Token Bucket): система має резервуар певної місткості (capacity), який заповнюється з фіксованою швидкістю (refillRate). Кожен виклик споживає 1 токен. Цей підхід дозволяє витримувати короткі сплески навантаження (bursts) з подальшим вирівнюванням швидкості.

Структура програмної реалізації

Рішення розміщено в робочій директорії Lab_4 та має таку архітектуру:

- throttle.ts — ядро утиліти: математика перевірки затримок, обслуговування Token Bucket, черга викликів FIFO, прив'язка таймерів Node.js та типи TypeScript.
- demo.ts — автономний демонстраційний скрипт, що імітує часові затримки, спам викликами та виводить форматовані мітки часу [+XXXms] у термінал.
- server.ts — HTTP-сервер для перевірки дроселя мережевими запитами за допомогою curl:
 - GET /throttle/window — демонстрація віконного ліміту (повертає HTTP 200 або HTTP 429).
 - GET /throttle/queue — демонстрація черги (повертає HTTP 202, а фонові завдання виконуються з обмеженням швидкості).
 - GET /throttle/token-bucket — перевірка поведінки кошика маркерів.

Тестування програмної реалізації

Перевірка реалізованого дроселя виконувалася комбінованим шляхом: за допомогою наочного консольного скрипта та через мережеві HTTP-запити.

1. Автономний консольний запуск (demo.ts)

Виконання здійснюється командою:

```

$ npx --yes tsx Lab_4/demo.ts

1. Window limit + drop strategy
[+001ms] executed call 1
[+001ms] executed call 2
[+001ms] executed call 3
[+053ms] result: 3 executed, 7 dropped

2. Leading versus trailing
[+055ms] leading-only: first
[+724ms] trailing-only: latest

3. Queue strategy
[+772ms] FIFO execution 1
[+773ms] FIFO execution 2
[+1776ms] FIFO execution 3
[+1777ms] FIFO execution 4
[+2784ms] FIFO execution 5
[+2785ms] FIFO execution 6

4. Cancellation
[+2989ms] cancelled pending call

```

Рис.1 – Автономний консольний запуск патерну throttle

Результати виконання сценаріїв:

- Сценарій 1 (Віконне обмеження + Drop): було відправлено 10 викликів одночасно. Зафіксовано виконання рівно 3 викликів, інші 7 були відкинуті (result: 3 executed, 7 dropped).
- Сценарій 2 (Leading vs Trailing): у режимі leading функція виконалася в момент [+055ms]; у режимі trailing початковий виклик проігноровано, а на відмітці [+724ms] виконався виклик з фінальним аргументом latest.
- Сценарій 3 (Черга FIFO): відправлено 6 викликів одночасно з лімітом 2/с. Запити виконувалися порціями по 2 виклики кожену секунду (відповідно на 770мс, 1770мс та 2780мс) зі збереженням точного порядку викликів від 1 до 6.
- Сценарій 4 (Скасування): запланований trailing-виклик було успішно скасовано через .cancel() до закінчення таймера, функція не викликала.

2. Ручна перевірка через HTTP-сервер (curl)

Запуск сервера:

```

$ npx --yes tsx Lab_4/server.ts
Throttle demo server listening at http://localhost:3000

```

Рис.2 – Старт сервера

Перевірка віконного відсікання:

```
$ npx --yes tsx Lab_4/server.ts
Throttle demo server listening at http://localhost:3000
[window-drop] executed value=1
[window-drop] executed value=2
[window-drop] executed value=3
[window-drop] executed value=9
[window-drop] executed value=10

$ for value in 1 2 3 4 5 6 7 8 9 10; do
  curl -s -o /dev/null -w "%{http_code}" "http://localhost:3000/throttle/window?value=$value"
done
printf "\n"
● 200 200 200 429 429 429 429 429 200 200
```

Рис.2 – Перевірка віконного відсікання

Результат: отримано послідовність статусів 200 200 200 429 429 429 429 429 200 200. На сервері оброблено рівно 3 звернення спочатку і як тільки відбулося відновлення – далі виконання нових запитів.

Перевірка буферизованої черги:

```
$ npx --yes tsx Lab_4/server.ts
Throttle demo server listening at http://localhost:3000
[window-drop] executed value=1
[window-drop] executed value=2
[window-drop] executed value=3
[window-drop] executed value=9
[window-drop] executed value=10
[queue] executed value=1
[queue] executed value=2
[queue] executed value=3
[queue] executed value=4
[queue] executed value=5
[queue] executed value=6

(main)
$ for value in 1 2 3 4 5 6; do
  curl -s "http://localhost:3000/throttle/queue?value=$value"
  printf "\n"
done
{"accepted":true,"strategy":"queue","value":"1","message":"Queued for FIFO execution; watch the server terminal."}
{"accepted":true,"strategy":"queue","value":"2","message":"Queued for FIFO execution; watch the server terminal."}
{"accepted":true,"strategy":"queue","value":"3","message":"Queued for FIFO execution; watch the server terminal."}
{"accepted":true,"strategy":"queue","value":"4","message":"Queued for FIFO execution; watch the server terminal."}
{"accepted":true,"strategy":"queue","value":"5","message":"Queued for FIFO execution; watch the server terminal."}
{"accepted":true,"strategy":"queue","value":"6","message":"Queued for FIFO execution; watch the server terminal."}
```

Рис.3 – Перевірка буферизованої черги

Результат: клієнт негайно отримує коди 202 Accepted, а в лозі сервера значення 1–6 з'являються поступово з інтервалами, що відповідають ліміту (по 2 за секунду).

Перевірка кошику токенів:

```
$ npx --yes tsx Lab_4/server.ts
[window-drop] executed value=2
[window-drop] executed value=3
[window-drop] executed value=9
[window-drop] executed value=10
[queue] executed value=1
[queue] executed value=2
[queue] executed value=3
[queue] executed value=4
[queue] executed value=5
[queue] executed value=6
[token-bucket] executed value=1
[token-bucket] executed value=2
[token-bucket] executed value=3

(main)
$ for value in 1 2 3 4 5; do
  curl -s -o /dev/null -w "%{http_code} %{url_effective}" "http://localhost:3000/throttle/token-bucket?value=$value"
done
200 http://localhost:3000/throttle/token-bucket?value=1
200 http://localhost:3000/throttle/token-bucket?value=2
200 http://localhost:3000/throttle/token-bucket?value=3
429 http://localhost:3000/throttle/token-bucket?value=4
429 http://localhost:3000/throttle/token-bucket?value=5
```

Рис.4 – Перевірка кошику токенів

Результат: отримано статуси 200 200 200 429 429. Перші 3 швидкі запити миттєво вичерпують початкову ємність кошика (capacity: 3). Наступні виклики відхиляються з кодом 429 Too Many Requests, оскільки токени ще не встигли відновитися відповідно до швидкості поповнення (refillRate).

Висновок

У ході виконання лабораторної роботи було досліджено принцип функціонування патерну Throttle (Дросель) та розроблено надійне, строго типізоване рішення мовою TypeScript.

Було реалізовано й успішно протестовано обидва ключові підходи до контролю навантаження: віконне лімітування (фіксація кількості подій за період) та алгоритм Token Bucket, що ефективно згладжує навантаження, дозволяючи короточасні сплески трафіку в межах заданої ємності. Розроблена підтримка стратегій переповнення довела свою працездатність: режим drop захищає систему шляхом відсікання надлишкових запитів (зі статусом 429), а режим queue забезпечує гарантовану доставку та черговість виконання без ризику перевантаження обробника. Завдяки комбінації прапорців leading та trailing, а також методам .cancel() та .flush(), створений модуль надає вичерпний контроль над життєвим циклом обробки подій.

Код програми

Код до даної лабораторної роботи розміщений у репозиторії https://github.com/MrCatman137/Cloud_Technologies у директорії Lab_4 та має повну інструкцію запуску даної програми.